OBJECT-ORIENTED PROGRAMMING • NRC 36862

LegalSync

Legal Case Management System

Team
CodeLex

Instructor
PhD. Jorge Edison Lascano

Institution
Universidad de las Fuerzas Armadas ESPE

First Partial • October 2026

CURRENT CHALLENGE

The Problem: A Law Firm Operating on Excel

Forgotten Deadlines

Excel does not notify when a hearing or court deadline is approaching.

Scattered Data

Cases, documents, and evidence live in separate files and isolated folders.

No Traceability

Impossible to identify which user modified a record or when changes occurred.

No Access Control

Everyone views and edits everything, without role-based security permissions.

PROJECT DEFINITION

Objective and Scope

MAIN OBJECTIVE

Centralize Law Firm Cases in a Java Application

Develop a console system using OOP that registers case files, controls access by role, notifies deadlines, and logs every system modification.

- Register and update cases with mandatory fields validation
- Link documents, physical evidence, and critical deadlines
- Trigger deadline alerts automatically upon login
- Search records by client name and audit via log.txt, CSV & TXT

First Partial Scope Limits:
NetBeans console application with TXT/CSV file persistence. GUI, MVC, and database integration are reserved for Unit 2.

REQUIREMENTS TRACEABILITY

Product Backlog: All 9 Requirements Implemented

ID	Requirement	Implementation Location in Code
PB-001	Roles & Permissions	`User.hasPermission(), Lawyer.restrictUserAccess()`
PB-002	Case Registration	`CaseFile.register(), validateMandatoryFields()`
PB-003	Document References	`Document stores path string reference`
PB-004	Deadline Alerts	`Deadline.calculateRemainingDays(), needsAlert()`
PB-005	Search by Client	`SearchEngine.searchByClientName()`
PB-006	Audit Logging	`AuditLog.registerEntry() → log.txt`
PB-007	Secure Login	`AuthenticationService.authenticate(), maskPassword()`
PB-008	CSV Import	`CsvImporter.importCases(), parseLine()`
PB-009	Document Generation	`DocumentGenerator.generate() → .txt`

SOFTWARE ARCHITECTURE

Project Structure: Modular Package Responsibilities

model

16 Diagram Classes

Contains domain entities: users, case files, deadlines, search engine, CSV importer, and audit logging.

utils

FileManager & Validator

Provides static utility methods for reading text files and validating input formats like ID numbers.

view

LegalSyncApp with main()

Handles user authentication, main interactive menu, and console input parsing via Scanner.

NetBeans Maven Project Structure:

Source Packages: 3 core packages | **Test Packages:** 6 JUnit 5 test classes

OBJECT ORIENTED DESIGN

Design: Final Class Diagram Architecture



14 Classes & 2 Enums

Complete implementation housed within the model package.

CaseFile as Central Domain Entity

Connects client, defendant, lawyer, documents, physical evidence, and legal deadlines.

100% Code Correspondence

The final diagram matches the submitted codebase exactly.

OOP PRINCIPLES

Inheritance: Lawyer and Legal Assistant extend User

```
public class Lawyer extends User {
  private String position;
  private String licenseNumber;

  public Lawyer(...) {
    super(userId, name, username,
          password, role);
    // Specific constructor logic
  }
}
```

Inherited from User

id, name, username, password, status, role along with login(), changePassword(), and hasPermission().

Lawyer Specialization

Adds position, licenseNumber, isPartner(), and restrictUserAccess().

LegalAssistant Specialization

Adds area, supervisor reference, and updateCaseData().

CLASS ARCHITECTURE

Class Relationships in Code

Association

Independent Entities

CaseFile references assigned Lawyer and Client. Both exist independently if the case is deleted.

Composition

Tightly Bound Lifecycle

Document, Evidence, and Deadline lists are instantiated inside CaseFile's constructor and die with it.

Aggregation

Shared Containers

SearchEngine indexes case files that exist externally: loadData(caseFiles).

Dependency

Transient Interaction

AuditLog uses User and CaseFile purely as method parameters in registerEntry() without storing them.

SYLLABUS APPLICATION

Unit 1 Core Topics Applied in LegalSync

Syllabus Topic	Implementation Example in LegalSync
Encapsulation	Private attributes; password changes strictly restricted to changePassword()
Constructors	CaseFile initializes with status OPEN, current date, and empty array lists
Getters & Setters	NetBeans auto-generated accessors: getName(), setStatus(), isActive()
Enums	Role and CaseStatus ensure type safety and eliminate typos
Collections	List using ArrayList, iterated seamlessly with for-each loops
File I/O	Scanner for reading console/files; FileWriter for outputting TXT and CSV
Static Methods	Validator.isValidIdNumber() and FileManager.readLines() for stateless operations

SECURITY CONTROL

Roles & Permissions Matrix (PB-001)

Operation	Partner	Lawyer	Junior	Assistant
Register, Update, Search Cases	✓Yes	✓Yes	✓Yes	✓Yes
Deadlines, Evidence, Documents	✓Yes	✓Yes	✕No	✕No
Import Data from CSV	✓Yes	✕No	✕No	✓Yes
Audit Logs & User Management	✓Yes	✕No	✕No	✕No

```
if (!currentUser.hasPermission("MANAGE_USERS")) { System.out.println("Access denied"); }
```

EXECUTION FLOW

System Workflow & Deadline Alerts (PB-004, PB-007)

1. Load users and cases from CSV
2. Secure login with max 3 attempts
3. Display automated deadline alerts
4. Interactive 13-option menu driven by permissions
5. Log every state change to log.txt

```
Username: atorres
Password entered: **********

Welcome Ana Torres (PARTNER)

===== DEADLINE ALERTS =====
[!] UPCOMING: Preliminary hearing, 3 day(s) remaining
[!] OVERDUE: Appeal submission, 5 day(s) ago
[!] UPCOMING: Mediation meeting, 5 day(s) remaining

---

Calculated via ChronoUnit.DAYS.between(LocalDate.now(), targetDate); negative values
trigger overdue status alerts.
```

ROBUSTNESS

File Persistence & Exception Handling

Storage Directory: `data/`

`credentials.txt`
Stores user accounts and assigned roles

`cases.csv`
Initial cases imported during startup

`log.txt`
Audit log populated via FileWriter append mode

`output/`
Directory where generated legal text files reside

Fault Tolerance Test: CSV Line 7 intentional invalid date is skipped gracefully, successfully loading remaining 5 records.

Specific try/catch Handling

`NumberFormatException`
Prevents crashes when non-numeric strings are parsed

`DateTimeParseException`
Catches invalid dates such as 2026-13-45

`IllegalArgumentException`
Handles unrecognized status strings outside enum bounds

`FileNotFoundException`
Manages missing configuration or data files cleanly

QUALITY ASSURANCE

JUnit 5 Unit Testing Results

76.92%

20 of 26 Tests Passing

Across 6 JUnit 5 Test Classes

Each class contains an intentional edge-case test failure (similar to divide(0,0) in BasicOperations) to identify boundaries.

Failing Test Case	Root Cause / Trigger Reason
`testMaskPasswordNull`	Null password input → throws NullPointerException
`testGetDetails`	Case missing assigned lawyer → NullPointerException
`testRegisterIdNumberWithDash`	"171234567-8": hyphen rejected as non-digit character
`testCalculateRemainingDaysWithoutDate`	Deadline missing date field → NullPointerException
`testSearchByClientNameEmpty`	contains("") evaluates to true, matching all 3 cases
`testLoginUppercaseUsername`	equals() check is strict case-sensitive

ITERATIVE DESIGN

Evolution: Refactorings from Design to Code

Architectural Refactoring	Design / Engineering Rationale
Added LAWYER role to Role enum	Lawyer actor existed in use case diagrams but lacked explicit enum mapping
Refactored status: String → CaseStatus	Strong typing via enum prevents invalid or misspelled state strings
Lawyer and LegalAssistant extend User	Replaced loose associations with proper OOP "is-a" inheritance relationships
Removed AuditEntry, Notification, AlertService	Simplified design: AuditLog stores formatted Strings directly; Deadline handles alerts
importFromCsv(filePath, lawyers) signature update	Required lawyer collection reference during import to establish case assignments
Corrected Typographical Errors	`Authentification → Authentication, Defedant → Defendant, Sutring → String`

SUMMARY

Conclusions

Solid OOP Design

Inheritance, association, aggregation, and composition applied effectively to solve a real-world problem.

Data Integrity

ArrayLists and TXT/CSV files secured with robust try/catch exception handling.

Quality Assurance

26 unit tests successfully identified 6 edge cases for future system optimization.

Questions?

LegalSync • CodeLex Team

Demo Credentials:

`atorres / partner123` | `cmena / lawyer123`